HACKEN

Blockchain Protocol Security Analysis Report

Customer: KiiChain

Date: 16/04/2026

We express our gratitude to the KiiChain team for the collaborative engagement that enabled the execution of this Blockchain Protocol Security Assessment.

KiiChain is a blockchain project focused on stablecoins and real-world assets, aimed at supporting practical financial use cases. Its broader goal is to improve utility, liquidity, and accessibility for users, businesses, and developers, particularly in emerging markets.

Document

Name	Blockchain Protocol Review and Security Analysis Report for KiiChain
Audited By	Tanuj Soni, Lipartiia Nino
Approved By	Ivan Bondar
Website	https://kiichain.io/
Changelog	10/04/2026 - Preliminary Report
Changelog	16/04/2026 - Final Report
Platform	KiiChain
Language	Golang
Tags	Cosmos SDK, Oracle, EVM-compatible
Methodology	https://docs.hacken.io/methodologies/blockchain-protocols

Review Scope

Repository	https://github.com/KiiChain/kiichain/
Commit	cfe29972b490324c3a919bf44dd4212fc1564e1a

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

18	16	2	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	1
Medium	1
Low	12

Vulnerability	Severity
F-2026-15610 - Sub-Second Remaining Duration Causes Division by Zero in CalculateReward	High
F-2026-15911 - Unpaginated DenomsFromAdmin Query Enables RPC Denial of Service	Medium
F-2026-15482 - Missing Base Token Price Validation May Cause Chain Halt	Low
F-2026-15484 - Lack of Validation for NativeOracleDenom Against Oracle Vote Targets	Low
F-2026-15485 - Outdated Governance Prices Cause Fee Token Mispricing	Low
F-2026-15613 - BeginBlock Panic From Rewards Pool And Bank Balance Mismatch	Low
F-2026-15621 - MsgUpdateParams Can Set Invalid Token Denom	Low
F-2026-15622 - Weak Genesis Validation in Rewards Module Allows Malformed State to be Persisted at Chain Start	Low
F-2026-15629 - Schedule Replacement Can Leave Previously Funded Rewards Undistributed	Low
F-2026-15670 - No Slippage Cap on Alternate-Token Fee Conversion	Low
F-2026-15692 - Imprecise End-Time Validation in ChangeSchedule Can Introduce a Halting State	Low
F-2026-15761 - Fee Conversion Uses Governance-Supplied Decimals Without Verifying Token Metadata	Low
F-2026-15890 - Documented Admin Revocation Path Is Unreachable	Low
F-2026-15910 - CosmWasm Metadata Path Bypasses Capability Check	Low
F-2026-15503 - Vote Targets Are Not Set at Genesis	Info
F-2026-15675 - Release Proportion Exceeds 1 When Block Time Surpasses EndTime	Info
F-2026-15685 - Nil Fee Tokens In Genesis Can Panic During Validation Or InitGenesis	Info

Vulnerability	Severity
F-2026-15959 - Missing Event Emission Across Core Modules	Info

Documentation quality

- All four custom modules have substantive README files covering state, messages, flows, and edge cases.
- Protobuf definitions have service- and message-level comments; field-level documentation would benefit from more consistent coverage.
- Additional committed documentation includes an EVM upgrade guide and a security vulnerability report in `contrib/docs/`. The project would benefit from formal specifications to capture design rationale.
- Godoc comments on exported keeper functions are present but brief — more detailed parameter and behavioral documentation would improve maintainability.
- The custom ante handler chain (23 Cosmos decorators, 2 EVM) is documented through inline comments; a standalone design document would help clarify the interaction between fee abstraction, oracle fee exemption, and the EVM path.

Code quality

- All custom modules have unit tests covering keepers, message servers, types, and genesis. The `ante/` package and fee abstraction ante decorators also have test coverage.
- The fee abstraction ante decorators are documented forks of the SDK's `DeductFeeDecorator` and the `cosmos/evm` mono decorator, with behavioral changes noted in each file's header.
- Simulation test support exists only for `tokenfactory`.
- 16 linters enabled (including `gosec`, `staticcheck`, `govet`, `errcheck`, `revive`) and enforced in CI.
- Error handling in ABCI hooks is sound — errors propagate to callers; non-critical failures are logged and skipped.
- A small number of `TODO` comments remain in production code.
- CI runs unit tests with coverage (Codecov) and linting on PRs. A separate e2e workflow runs on pushes to `main` and PRs targeting `main`.

Architecture quality

- **Decentralization:** No sudo or admin-privileged module exists. All custom module governance operations route authority through `x/gov`. Oracle votes are restricted to bonded validators or their delegated feeders. `MsgFundPool` and `MsgCreateDenom` are callable by any account; tokenfactory mint, burn, force transfer, and metadata operations require per-denomination admin status.
- **Resilience:** Fee abstraction self-disables when the native denomination's oracle TWAP becomes unavailable, and independently disables individual fee tokens with missing TWAPs. The oracle handles empty ballot periods gracefully. Price clamping limits per-block fee token price movement.
- **Interoperability:** Custom CosmWasm plugins expose oracle, EVM, tokenfactory, and bech32 queries to contracts, and tokenfactory message execution. A custom oracle EVM precompile exposes exchange rates to Solidity contracts. The IBC transfer stack is extended with Packet Forward Middleware and rate limiting.
- **Module dependencies:** Fee abstraction depends on oracle, ERC20, and bank keepers at runtime — failures can cascade into fee processing, partially mitigated by self-disabling. The rewards module must remain first in `BeginBlocker` order to feed coins to distribution.

Table of Contents

System Overview

KiiChain is a Cosmos SDK appchain built on CometBFT consensus with two integrated smart contract runtimes: an EVM and CosmWasm. IBC-Go provides cross-chain communication. The daemon binary is `kiichaind`, the native denomination is `akii`.

The chain extends the standard Cosmos module set with four custom modules, and bridges native functionality into both contract runtimes via EVM precompiles and CosmWasm custom plugins.

Custom Modules

- **Oracle** (`x/oracle`)
  Decentralized price feed. Validators or delegated feeders submit aggregate exchange rate votes via `MsgAggregateExchangeRateVote`. Feeder delegation is managed through `MsgDelegateFeedConsent`. The keeper depends on `AccountKeeper`, `BankKeeper`, and `StakingKeeper`.
  EndBlock (on the last block of each `VotePeriod`) tallies submitted ballots by denomination against a reference denom, persists weighted-median exchange rates, records per-validator success/miss/abstain counts, clears votes, applies the whitelist, and writes a `PriceSnapshot`. BeginBlock (on the last block of each `SlashWindow`) slashes validators who missed vote obligations and prunes stale exchange rates. Two ante decorators — `VoteAloneDecorator` and `SpammingPreventionDecorator` — enforce oracle transaction rules.
- **Token Factory** (`x/tokenfactory`)
  Permissionless denomination creation and management. Any account can create native coins under `factory/{creator}/{subdenom}` via `MsgCreateDenom`. The denom admin can mint (`MsgMint`), burn (`MsgBurn`), force-transfer (`MsgForceTransfer`), change admin (`MsgChangeAdmin`), and set bank metadata (`MsgSetDenomMetadata`). Burn-from, force-transfer, and set-metadata operations are gated by `enabledCapabilities` on the keeper. The keeper depends on `AccountKeeper`, `BankKeeper`, and `CommunityPoolKeeper`. No BeginBlock or EndBlock logic. This is the only custom module exposed to CosmWasm contracts for both queries and message execution.
- **Fee Abstraction** (`x/feeabstraction`)
  Allows fees to be paid in non-native tokens. The keeper depends on `BankKeeper`, `Erc20Keeper`, and `OracleKeeper`. BeginBlock fetches TWAPs from the oracle over a configured lookback window and updates per-fee-token prices (with clamping); if the native oracle denom has no valid TWAP, the module disables itself by writing `Enabled = false` to params.
  At transaction time, custom ante decorators (one for Cosmos txs, one for EVM txs) call `ConvertNativeFee`. This checks whether the user has sufficient native balance; if not, it iterates enabled fee tokens, computes the ceiling-rounded equivalent amount, and attempts to source it from the user's bank balance or via ERC20-to-native conversion through the `erc20Keeper`. The first successful token is used, and a `convert_fees` event is emitted. Governed by `MsgUpdateParams` and `MsgUpdateFeeTokens`.
- **Rewards** (`x/rewards`)
  Time-based linear reward distribution. Stores a `RewardPool`, a `ReleaseSchedule` (with `TotalAmount`, `ReleasedAmount`, `EndTime`, `LastReleaseTime`, `Active`), and `Params`. BeginBlock computes a linear proportion of remaining rewards based on elapsed seconds, sends that amount from the rewards module account to the fee collector via `SendCoinsFromModuleToModule`, and updates the schedule. In

the begin block ordering, `rewards` runs before `distribution`, ensuring released coins are included in the standard validator/delegator distribution. Anyone can fund the pool via `MsgFundPool`; schedule and params are governance-controlled.

EVM Precompiles

Static precompiles extend the Berlin set with: `bech32` and `p256` (stateless), plus `staking`, `distribution`, `ics20`, `bank`, `gov`, `slashing`, and a custom `oracle` precompile (stateful). These are registered in `app/keepers/precompiles.go` and give Solidity contracts direct access to native chain operations.

CosmWasm Bindings

Custom plugins registered in `wasmbinding/` expose four query types to contracts — `TokenFactory`, `EVM`, `Bech32`, `Oracle` — and one message type — `TokenFactory` (create, mint, burn, transfer, change admin, set metadata).

IBC Stack

Assembled in `app/keepers/keepers.go`. The transfer stack layers Rate Limiting over Packet Forward Middleware over IBC Callbacks over the base transfer module. The ICA controller is wrapped with IBC Callbacks; the ICA host runs standalone. A wasm IBC handler is registered on its own port and serves as the contract keeper for callback dispatch. A separate IBCv2 transfer stack with callbacks and rate limiting is also configured.

Risks

- The fee abstraction module operates without user-facing controls over token selection. When a user's native balance is insufficient to cover transaction fees, the system automatically iterates the fee token registry in a fixed order and selects the first enabled token for which the user holds a sufficient balance — either as a bank denomination or as an ERC20 token, converting the latter without explicit user consent. There is no mechanism to opt out of fee abstraction on a per-transaction basis, specify a preferred fee token, exclude specific tokens, or set a maximum acceptable fee amount. This may result in unexpected expenditure of tokens the user intended to hold, with the risk amplified during periods of rapid price movement when per-block oracle prices lag behind actual market rates. See F-2026-15670 for more context.

Findings

Vulnerability Details

F-2026-15610 - Sub-Second Remaining Duration Causes Division by Zero in CalculateReward - High

Description:

The `CalculateReward` function can panic due to insufficient validation of the remaining release duration, which may result in a division by zero.

Specifically, the implementation checks only whether `schedule.EndTime` is exactly equal to `schedule.LastReleaseTime`, assuming this is sufficient to prevent a zero denominator:

x/rewards/types/reward.go

```
// If total duration would be 0, there would be a div by 0
if schedule.EndTime.Equal(schedule.LastReleaseTime) {
return sdk.Coin{}, fmt.Errorf("end time is equal to last release an
d would do a division by 0. EndTime: %s", schedule.EndTime)
}
...
totalDurationStamp := schedule.EndTime.Sub(schedule.LastReleaseTime
) // Remaining release period
...
totalDuration := math.NewInt(int64(totalDurationStamp.Seconds())).B
igInt()
...
// Calculate linear release proportion between 0 and 1
releaseProportion := math.LegacyNewDecFromBigInt(timeElapsed).Quo(m
ath.LegacyNewDecFromBigInt(totalDuration))
```

However, this validation is incomplete. Although `totalDurationStamp` is guaranteed to be non-zero by the equality check, `int64(totalDurationStamp.Seconds())` truncates fractional seconds. As a result, if `totalDurationStamp` is greater than `0` but less than `1` second, it will be converted to `0`.

Since `totalDuration` is not validated after this conversion, `releaseProportion` may perform a division by zero and panic. Because this logic is executed from `BeginBlocker`, such a panic can halt block production and stop the chain.

Notably, this issue does not require malicious input. It can occur whenever block execution falls into the interval `[EndTime - 1s, EndTime)`, causing the next `BeginBlocker` execution to panic. While the issue is timing-dependent and may not trigger immediately, it appears realistically reachable over a long-running chain operation.

Assets:

- x/rewards [https://github.com/KiiChain/kiichain/tree/main/x/rewards]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Severity: High

Recommendations

Remediation: Consider replacing the current seconds-based truncation in `types/reward.go` with integer nanosecond arithmetic. This would preserve sub-second precision and prevent positive sub-second durations from being converted to zero. Additionally, validate that the final denominator remains strictly greater than zero before performing the division.

Resolution: The issue has been resolved in the referenced pull request.

Evidences

Test Panic Scenario in CalculateReward

Reproduce:

Add test to `./x/rewards/types/reward_test.go`.

```
// TestCalculateReward_PanicScenario simulates the block-by-block
// sequence that triggers the division-by-zero in CalculateReward:
Block N lands
// within 1 second of EndTime, Block N+1 hits the panic.
func TestCalculateReward_PanicScenario(t *testing.T) {
denom := "akii"
endTime := time.Date(2025, 6, 15, 12, 0, 0, 0, time.UTC)

// Initial state: 500 remaining, last release was ~6s before EndTim
e
schedule := types.ReleaseSchedule{
TotalAmount: sdk.NewCoin(denom, math.NewInt(1000)),
ReleasedAmount: sdk.NewCoin(denom, math.NewInt(500)),
LastReleaseTime: endTime.Add(-6200 * time.Millisecond),
EndTime: endTime,
Active: true,
}

// Block N: blockTime = EndTime - 700ms (lands in the danger zone)
blockN := endTime.Add(-700 * time.Millisecond)

resultN, err := types.CalculateReward(blockN, schedule)
require.NoError(t, err, "block N should succeed normally")
require.True(t, resultN.Amount.GT(math.ZeroInt()), "block N should
release something")

// Simulate BeginBlocker state update
```

```
schedule.ReleasedAmount = schedule.ReleasedAmount.Add(resultN)
schedule.LastReleaseTime = blockN

// Confirm remaining > 0 (proportion < 1 because blockTime < EndTim
e)
remaining := schedule.TotalAmount.Amount.Sub(schedule.ReleasedAmoun
t.Amount)
require.True(t, remaining.GT(math.ZeroInt()),
"remaining must be > 0 for the panic to trigger (got %s)", remainin
g)

// Block N+1: 5 seconds later → triggers division by zero
blockN1 := blockN.Add(5 * time.Second)

require.Panics(t, func() {
_, _ = types.CalculateReward(blockN1, schedule)
}, "block N+1 should panic due to totalDuration truncating to 0")
}
```

Run the test:

```
go test ./x/rewards/types/ -run "TestCalculateReward_PanicScenario"
```

Results:

```
ok github.com/kiichain/kiichain/v7/x/rewards/types 0.025s
```

F-2026-15911 - Unpaginated DenomsFromAdmin Query Enables RPC Denial of Service - Medium

Description:

`GetDenomsFromAdmin` scans the entire creator-prefix store, covering every factory denom across all creators, and performs one `GetAuthorityMetadata` lookup per denom to compare its `Admin` field.

This behavior creates a denial-of-service risk due to the combination of three properties:

1. **Chain-wide linear complexity.** The function has `O(D)` cost, where `D` is the total number of tokenfactory denoms on the chain. For each denom, it performs additional store access and protobuf decoding to retrieve and inspect the authority metadata.
2. **No pagination.** The query interface does not support `PageRequest` / `PageResponse`, so the full result set is collected in memory and returned in a single response.
3. **No gas-based cost control.** Cosmos SDK gRPC queries are executed outside the transaction gas model, so the requester pays no economic cost regardless of how much work the node performs.

As a result, an attacker can repeatedly query `DenomsFromAdmin` against a node's gRPC or REST endpoint using any address, including `""`, which matches all denoms whose admin has been renounced. As the number of tokenfactory denoms grows over time, each request becomes progressively more expensive to serve.

Note that similar efficiency concerns also apply to `DenomsFromCreator`. However, its impact is more limited because the iteration is scoped to a single creator's denoms rather than the full chain-wide denom set, as in `GetDenomsFromAdmin`.

This can degrade RPC responsiveness and may be used to cause denial of service on public-facing RPC nodes. The impact increases naturally with chain adoption, as more denoms are created.

Assets:

- x/tokenfactory [https://github.com/KiiChain/kiichain/tree/main/x/tokenfactory]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation: Refactor `GetDenomsFromAdmin` to avoid chain-wide iteration. Prefer maintaining a secondary index from admin address to denom entries, updated whenever `setAuthorityMetadata` changes a denom's admin. This would make lookups scale with the number of denoms controlled by the queried admin instead of the total number of factory denoms on the chain. Note that this requires a one-time state migration to backfill the index for existing denoms.

As an additional measure, add `cosmos.base.query.v1beta1.PageRequest` / `PageResponse` support for both `GetDenomsFromAdmin` and `DenomsFromCreator` and enforce a hard maximum page size so that each query performs bounded work and returns a bounded result set.

Resolution: The issue has been resolved in the referenced pull request.

F-2026-15482 - Missing Base Token Price Validation May Cause Chain Halt - Low

Description:

The `CalculateFeeTokenPrices` function does not explicitly validate whether the base token price is zero. As a result, a zero value may be propagated to `calculatePriceTokens`, where prices for other fee tokens are derived.

This leads to a failure in the following logic:

```
// Calculate the price of the token in terms of the base token
price, err := types.CalculateTokenPrice(baseTokenPrice, tokenPrice)
if err != nil {
return nil, errorsmod.Wrapf(sdkerrors.ErrInvalidRequest, "error cal
culating token price for denom %s: %v", token.Denom, err)
}
```

If `baseTokenPrice` is zero, `CalculateTokenPrice` returns an error, which is then propagated back to `CalculateFeeTokenPrices` and further up to the `BeginBlocker`. Since this occurs during block processing, it can cause the chain to halt.

Although the likelihood of the base token price being zero is low, the impact is high, as it directly affects chain liveness.

Assets:

- x/feeabstraction [https://github.com/KiiChain/kiichain/tree/main/x/feeabstraction]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

Consider adding an explicit validation to ensure `baseTokenPrice` is non-zero before performing price calculations:

```
// Find the price for the base token
baseTokenPrice, ok := twapPriceMap[params.NativeOracleDenom]
if !ok || !baseTokenPrice.IsPositive() {
// Disable fee abstraction if there is no pricing
k.Logger(ctx).Debug("%s has no price, feeabstraction disabled", par
ams.NativeOracleDenom)
```

```
params.Enabled = false
return k.Params.Set(ctx, params)
}
```

Resolution: The issue has been resolved in the referenced pull request.

F-2026-15484 - Lack of Validation for NativeOracleDenom Against Oracle Vote Targets - Low

Description:

In `UpdateParams` in the fee abstraction module, `NativeOracleDenom` is only validated to ensure it is a syntactically valid denom. However, there is no verification that it is among the denoms registered as oracle vote targets.

As a result, an invalid `NativeOracleDenom` (i.e., not supported by the oracle) can be set. In such a case, the fee abstraction module will be disabled at the beginning of the next block, as fee token price calculation becomes impossible. While this behavior is expected given the configuration, the issue stems from missing validation at the parameter update stage.

Validating that `NativeOracleDenom` is included in the set of oracle vote target denoms during `UpdateParams` would provide earlier failure detection and reduce the risk of misconfiguration.

Assets:

- x/feeabstraction [https://github.com/KiiChain/kiichain/tree/main/x/feeabstraction]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Introduce validation in `UpdateParams` to verify that `NativeOracleDenom` is among the oracle vote targets (`ms.oracleKeeper.GetVoteTargets()`), and reject the update if the check fails.

Resolution: The issue has been resolved in the referenced pull request.

F-2026-15485 - Outdated Governance Prices Cause Fee Token Mispricing - Low

Description:

`UpdateFeeTokens` requires governance proposals to include a fixed token price at submission time. However, proposals undergo a deposit and voting period before execution. By the time a proposal is enacted, the submitted price is likely stale, as the actual TWAP may have diverged significantly during the voting window.

Once this stale price is written to the state, `CalculateFeeTokenPrices` applies per-block clamping via `ClampPrice(token.Price, newTwapPrice, clampFactor)`. Since the clamping mechanism does not distinguish between governance-provided prices and regular price values, convergence to the current TWAP is artificially rate-limited to ± `clampFactor` per block.

Note that `ClampPrice` bypasses clamping when `prevPrice == 0`. However, validation rules in `UpdateFeeTokens` prohibit setting `Price == 0`, meaning governance has no way to signal "adopt the current TWAP immediately" and bypass clamping.

This behavior is not the result of malicious governance, but rather an inherent limitation of the proposal lifecycle: it enforces the use of a price that is likely outdated at execution time. As a result, each `UpdateFeeTokens` proposal carries a risk of introducing temporarily inaccurate pricing.

Assets:

- x/feeabstraction [https://github.com/KiiChain/kiichain/tree/main/x/feeabstraction]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

Consider modifying the `UpdateFeeTokens` logic so that prices provided in `types.MsgUpdateFeeTokens` do not directly influence fee token pricing in subsequent blocks.

Instead, governance updates should avoid setting an explicit price and rely on oracle-derived TWAP values during `BeginBlocker`. One possible approach is to initialize token prices to `0`, allowing `ClampPrice` to bypass clamping and adopt the current TWAP immediately.

Alternatively, introduce a mechanism to explicitly signal that the current TWAP should be used as the initial price, avoiding reliance on stale governance-provided values.

Resolution: The issue has been resolved in the referenced PR#322 and PR#236.

F-2026-15613 - BeginBlock Panic From Rewards Pool And Bank Balance Mismatch - Low

Description:

In `x/rewards/keeper/abci.go`, `BeginBlocker` first calls `SendCoinsFromModuleToModule`, which moves coins based on the real balance of the rewards module account in the bank module. It then updates `RewardPool.CommunityPool` with `CommunityPool.Sub(...)`, which only reflects stored decimal accounting in the application state. The code does not verify that `CommunityPool` is large enough for the release before the transfer. If bank balance and `CommunityPool` diverge (module has enough coins on the bank side, but accounting is too low—or the release denom is missing from `CommunityPool` while the bank still has funds), the transfer can succeed while the subtract would drive a denom below zero, and Cosmos SDK `DecCoins.Sub` then panics with a `negative coin amount`.

The rewards module's bank balance and `CommunityPool` can diverge if coins move without updating the pool. This can occur due to a mismatched genesis configuration or direct transfers to the module account. Normally, `MsgFundPool` and `FundCommunityPool` keep them aligned, but any deviation from this flow breaks the invariant.

If a panic occurs in `BeginBlock`, it can halt the chain or cause block processing to fail. This is not a user-level exploit but a system integrity issue, where broken invariants lead to a failure instead of a recoverable error.

File: *x/rewards/keeper/abci.go:*

```
if err := k.bankKeeper.SendCoinsFromModuleToModule(ctx, types.Modul
eName, k.feeCollectorName, coinsToDistribute); err != nil {
return err
}
rewardPool.CommunityPool = rewardPool.CommunityPool.Sub(sdk.NewDecC
oinsFromCoins(coinsToDistribute...))
```

Cosmos SDK `types.DecCoins.Sub` panics when subtraction would yield a negative amount for any denom.

Assets:

- x/rewards [https://github.com/KiiChain/kiichain/tree/main/x/rewards]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Before performing the transfer, verify that the `CommunityPool` balance for the configured denom is sufficient to cover `coinsToDistribute`, and return an error otherwise.
- Prefer using `SafeSub`, and handle a negative result explicitly by returning an error instead of calling `Sub` directly.
- During genesis validation or initialization, optionally verify that the rewards module account balance for the configured token denom matches the corresponding `CommunityPool` balance.

Resolution: The issue has been resolved in the referenced pull request.

F-2026-15621 - MsgUpdateParams Can Set Invalid Token Denom - Low

Description:

`MsgUpdateParams` accepts a `Params` object where `token_denom` is validated only for a non-empty value and not for SDK denom-format correctness. Because `UpdateParams` persists params immediately after this weak check, governance can set a syntactically invalid denom in module state. Once this happens, normal reward operations can fail in practice because other flows still rely on valid coin validation rules, so operators may be unable to use reward funding and schedule changes normally until a corrective governance action updates params again.

`x/rewards/types/params.go`:

```
func (p Params) ValidateBasic() error {
denom := p.TokenDenom
if denom == "" {
return fmt.Errorf("invalid denom, empty: %s", denom)
}
return nil
}
```

`x/rewards/keeper/msg_server.go`:

```
if err := msg.Params.ValidateBasic(); err != nil {
return nil, err
}

if err := k.Params.Set(ctx, msg.Params); err != nil {
return nil, err
}
```

This issue can create persistent operational denial of service for reward administration because invalid params can make common reward flows unusable until governance fixes the value. It does not provide a direct third-party fund theft path, but it can still interrupt expected reward lifecycle operations and delay protocol administration.

Assets:

- x/rewards [https://github.com/KiiChain/kiichain/tree/main/x/rewards]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Consider validating `TokenDenom` with the standard `ValidateDenom` helper instead of only checking whether the value is non-empty. This would ensure that the denom complies with the expected format requirements, including valid length bounds and allowed characters, and would prevent malformed values from being accepted through `UpdateParams`.

Resolution: The issue has been resolved in the referenced pull request.

F-2026-15622 - Weak Genesis Validation in Rewards Module Allows Malformed State to be Persisted at Chain Start - Low

Description:

`GenesisState.Validate()` delegates to three independent sub-validators but enforces neither the complete `sdk.DecCoins` structural invariants on `RewardPool.CommunityPool` nor cross-component denom consistency against `Params.TokenDenom`. Both gaps admit malformed genesis states that silently pass all checks and are then persisted verbatim by `InitGenesis`, becoming latent defects that only surface at runtime.

```
func (gs *GenesisState) Validate() error {
if err := gs.Params.ValidateBasic(); err != nil {
return err
}

if err := gs.RewardPool.ValidateGenesis(); err != nil {
return err
}
return gs.ReleaseSchedule.ValidateGenesis()
}
```

1. Incomplete `DecCoins` invariants on `CommunityPool`

```
func (rp RewardPool) ValidateGenesis() error {
if rp.CommunityPool.IsAnyNegative() {
return fmt.Errorf("negative CommunityPool in distribution fee pool,
is %v",
rp.CommunityPool)
}

return nil
}
```

`IsAnyNegative()` only excludes negative amounts. Three additional invariants are enforced by `sdk.DecCoins.Validate()` are left unchecked:

- Valid denom strings — arbitrary byte sequences are accepted.
- Canonical lexicographic ordering — out-of-order coins pass.
- No duplicate denoms — the same denom may appear more than once.

2. No cross-component denom consistency check against `Params.TokenDenom`

`GenesisState.Validate()` runs `Params`, `RewardPool`, and `ReleaseSchedule` validators in isolation. None of them compares their denom fields against `Params.TokenDenom`, so two mismatches can be admitted at genesis with distinct runtime consequences.

2.1. `ReleaseSchedule.TotalAmount.Denom` ≠ `Params.TokenDenom`.

The module account is expected to hold only `TokenDenom` coins. However, `CalculateReward()` constructs the reward coin using the denom taken directly

from `ReleaseSchedule.TotalAmount.Denom`, and `BeginBlocker` then attempt to send that coin from the module account. If this denom differs from `Params.TokenDenom`, the module account will not hold the required balance, causing the send to fail with an insufficient-funds error on every block starting from block 2. This permanently breaks reward distribution.

2.2. `CommunityPool` denoms ≠ `Params.TokenDenom`.

If the community pool is initialized with a different denom while `ReleaseSchedule.TotalAmount.Denom` is correctly set to `TokenDenom`, the reward transfer in `BeginBlocker` succeeds, but the subsequent deduction from the community pool operates on an unexpected denom and panics. As a result, the chain can enter a runtime-failure condition despite passing genesis validation.

Neither gap is accessible to external users at runtime. The `FundPool` and `ChangeSchedule` message handlers both enforce `params.TokenDenom` equality and legitimate pool operations always produce canonical `DecCoins`. The risk is a chain startup failure or a corrupt-but-live initial state that produces difficult-to-diagnose failures later.

Assets:

- x/rewards [https://github.com/KiiChain/kiichain/tree/main/x/rewards]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: `RewardPool.ValidateGenesis` should perform full `DecCoins` validation in addition to non-negativity checks so malformed entries are rejected at genesis time. Add targeted genesis tests for invalid denom formats, duplicate denoms, and non-canonical ordering, and keep this validation aligned with Cosmos SDK coin invariants to prevent future drift.

Additionally, `GenesisState.Validate` should enforce denom consistency across all three components after each sub-validator passes. Specifically, every coin in `CommunityPool` must carry `Params.TokenDenom`, and when the schedule is active, `ReleaseSchedule.TotalAmount.Denom` must equal `Params.TokenDenom`. Add targeted genesis tests covering an active schedule with a mismatched `TotalAmount` denom, a pool seeded with a foreign

denom, and both mismatches combined, to confirm that none of these states can reach `InitGenesis`.

Resolution: The issue has been resolved in the referenced pull request.

F-2026-15629 - Schedule Replacement Can Leave Previously Funded Rewards Undistributed - Low

Description:

The rewards module allows the schedule to be changed once the change has been approved through governance. When `ChangeSchedule` is executed, it replaces the previous schedule immediately, without considering whether the previous schedule is still active and expected to continue distributing funds.

This behavior is not inherently incorrect, but it may lead to the following edge-case scenario:

1. While the current schedule is still active, an `UpdateParams` proposal is executed, and the reward token denom is changed from `denom1` to `denom2`. The existing schedule continues to operate correctly and keeps distributing rewards in `denom1`.
2. The pool is then funded with `denom2` tokens via `FundPool`.
3. Before the original schedule finishes, a `ChangeSchedule` proposal is executed with a new schedule that is intended to distribute `denom2` instead of `denom1`.
4. As a result, the old schedule is replaced, and reward distribution starts using `denom2`. At the same time, the remaining `denom1` tokens already held by the rewards module are no longer referenced by any active schedule and become stuck, with no direct mechanism to recover or redistribute them.

The only apparent way to use the remaining `denom1` funds is to go through the same governance-driven process again: change the rewards denom back from `denom2` to `denom1` and update the schedule so that the leftover tokens can be distributed. However, this recovery path is not ideal for several reasons:

- it depends on a successful governance vote and therefore is not guaranteed;
- it may introduce unfairness into the intended distribution process, since previously allocated funds could be distributed much later than originally intended;
- The value of `denom1` may materially change over time, causing the delayed distribution to have a different economic effect than originally expected.

As a result, previously funded tokens in the old denom may remain stranded in the rewards module if the active schedule is replaced before those funds are fully distributed.

Assets:

- x/rewards [https://github.com/KiiChain/kiichain/tree/main/x/rewards]

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation: To address this issue, the project should first clarify the intended behavior around reward denom transitions and schedule replacement, and then align the implementation and documentation with that design. In particular, the following measures should be considered:

- Clearly define and document the expected behavior of `ChangeSchedule`, particularly whether a newly approved schedule should take effect immediately when a previous schedule is still active.
- Define and simplify the intended denom-switch procedure. The current flow is operationally fragile because it depends on a specific sequence of actions (`UpdateParams` → `FundPool` → `ChangeSchedule`) and may fail or strand funds if performed differently.
- Update the implementation so that replacing a schedule or changing the reward denom cannot leave previously funded reward tokens stranded in the module.
- Additionally, document that the rewards module's `QueryParamsRequest` gRPC response reflects the denom configured for future reward behavior, which may differ from the denom used by the currently active schedule.
- Depending on the intended design, consider deferring activation of a new schedule until the previous one is exhausted, or introducing an explicit mechanism to recover or otherwise account for undistributed funds in the previous denom.

Overall, the fix should ensure that schedule and denom transitions are predictable, well documented, and do not result in reward funds becoming stranded or economically misaligned.

F-2026-15670 - No Slippage Cap on Alternate-Token Fee Conversion - Low

Description:

The fee abstraction module converts a native-denominated transaction fee into an alternate fee token using the `FeeTokens` state loaded during ante fee deduction. That state is updated in `BeginBlock` from oracle TWAPs and governance-controlled parameters.

The protocol does not bind this conversion to the price observed at signing time, first gossip, `CheckTx`, or `RecheckTx`. It does not provide a TTL or expiry height for a quoted conversion rate, and it does not allow the user to specify a maximum alternate-token debit, slippage bound, or similar protection for the non-native fee leg. As a result, the final amount of alternate token charged at `DeliverTx` can differ significantly from what an off-chain estimator or an earlier mempool check implied, especially if inclusion is delayed or prices move.

Consequently, a user who lacks sufficient native gas token but holds a supported alternate fee token may sign a transaction that appears affordable at `CheckTx` time based on the then-current `FeeTokens` price. If the transaction remains in the mempool and is included later, `BeginBlock` may update the stored conversion price before `DeliverTx` executes. As a result, the user is charged a larger amount of the alternate token at inclusion time than the price at `CheckTx` implied, with no protocol mechanism to reject or cap that overage.

From a security-classification perspective, this behavior does not by itself amount to unauthorized transfer of third-party assets or chain halt. It is an oracle- and policy-dependent economic exposure, analogous to the familiar case where fees at inclusion differ from simulation, extended here to the amount charged in an alternate fee denom.

Assets:

- x/feeabstraction [https://github.com/KiiChain/kiichain/tree/main/x/feeabstraction]

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation: Documentation should state explicitly that alternate-token fee amounts are determined at inclusion height from module state after `BeginBlock` price updates. Wallet and API layers should label fee previews as non-binding, prefer a query of `FeeTokens` plus simulation at the chain tip immediately before broadcast, and refresh estimates if broadcast is retried after a delay.

If product requirements demand stronger user protection, the protocol could consider an explicit max alternate-fee coin or a bound on premium versus a committed quote enforced in ante. Such a change would require careful analysis of replay, minimum gas price, and EVM versus Cosmos parity. Separately, governance may tune `TwapLookbackWindow` and `ClampFactor` to trade responsiveness against maximum per-block price step, but that tuning does not replace a per-transaction user limit.

F-2026-15692 - Imprecise End-Time Validation in ChangeSchedule Can Introduce a Halting State - Low

Description:

The rewards module does not strictly enforce that a new schedule's `EndTime` must be in the future. In particular, the current validation allows `EndTime == ctx.BlockTime()` to pass during `ChangeSchedule`.

As a result, governance can approve and apply a schedule whose time parameters are internally inconsistent for future reward processing. Specifically, it is possible to set a schedule with:

- `EndTime == LastReleaseTime == ctx.BlockTime()`
- `ReleasedAmount < TotalAmount`
- `Active = true`

Such a schedule passes validation and becomes active. However, on the next block, `BeginBlocker` invokes `CalculateReward`, which rejects the state when `EndTime == LastReleaseTime` and returns an error:

```
func CalculateReward(blockTime time.Time, schedule ReleaseSchedule)
(sdk.Coin, error) {
...
// If total duration would be 0, there would be a div by 0
if schedule.EndTime.Equal(schedule.LastReleaseTime) {
return sdk.Coin{}, fmt.Errorf("end time is equal to last release an
d would do a division by 0. EndTime: %s", schedule.EndTime)
}
```

This causes `BeginBlocker` to return an error and halt block production.

The problematic state is primarily reachable through `ChangeSchedule`. A secondary edge case also exists in the normal execution flow: if `LastReleaseTime` is initially zero, `BeginBlocker` sets it to the current block time and returns; if that timestamp happens to exactly match `EndTime`, the same invalid state may be reached on the following block:

```
// If active and there is no previous time stamp, set it as current
block's and skip this time
if schedule.LastReleaseTime.IsZero() {
schedule.LastReleaseTime = ctx.BlockTime()
return k.ReleaseSchedule.Set(ctx, schedule)
}
```

This can likewise cause `BeginBlocker` to return an error and halt block production.

In practice, however, this issue is extremely unlikely to be triggered and remains difficult to exploit meaningfully even under malicious governance, as it requires a very specific schedule configuration and timing alignment.

Assets:

- x/rewards [https://github.com/KiiChain/kiichain/tree/main/x/rewards]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: To address this issue, the implementation should avoid propagating this edge-case failure from `CalculateReward` into `BeginBlocker`. Instead, if the schedule has reached a terminal state where `EndTime == LastReleaseTime` while rewards still remain, the module should handle it gracefully by releasing the remaining rewards directly and deactivating the schedule, rather than returning an error that halts block production.

In addition, the schedule time validation should be tightened to reduce the likelihood of such states being introduced in the first place. In particular, `validateEndTime` should enforce not only that `EndTime` is in the future, but also that the remaining duration is above some minimum threshold.

Resolution: PR #299 removes the `CalculateReward` halt path (`EndTime <= LastReleaseTime` now releases remaining rewards instead of erroring), while stricter end-time buffering is an accepted governance-gated design choice.

F-2026-15761 - Fee Conversion Uses Governance-Supplied Decimals Without Verifying Token Metadata - Low

Description:

The conversion path `convertERC20ForFees` uses `types.CalculateTokenAmountWithDecimals` with `feePrice.Decimals` taken from `FeeTokenMetadata`, which is set or updated via governance messages and genesis. Validation only enforces that decimals are an integer between 1 and 18 and that the denom strings are well-formed. The message server does not read the ERC-20 contract's `decimals()`, bank denom metadata, or the erc20 module's pair record to confirm that the registered value matches live token precision.

If governance registers wrong decimals for a paired asset, every conversion using that entry scales the alternate-token amount incorrectly. Depending on direction, users may be overcharged (pay more alternate token than intended relative to oracle price) or undercharged (risk insolvency or fee shortfall to validators if combined with other assumptions), or hit dust rounding to zero and skip tokens unexpectedly. Because the error is stateful and deterministic, it affects all transactions using that fee token until the registry is corrected.

Assets:

- x/feeabstraction [https://github.com/KiiChain/kiichain/tree/main/x/feeabstraction]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

On `MsgUpdateFeeTokens` (and optionally genesis validation where pairs exist), resolve the token's canonical decimals from the erc20 pair or bank metadata and reject proposals where `FeeTokenMetadata.Decimals` differs. Add unit tests for a deliberate mismatch. Wallets can display a warning when queried metadata disagrees with the on-chain contract.

Resolution:

The issue is substantively resolved by this pull request

F-2026-15890 - Documented Admin Revocation Path Is Unreachable - Low

Description:

As documented in the tokenfactory module README, `ChangeAdmin` is intended to support changing the master admin account, including setting it to `""` to revoke admin rights entirely so that no account retains administrative control over the asset.

In practice, this documented behavior is unreachable because `MsgChangeAdmin.ValidateBasic()` rejects an empty `NewAdmin` value:

```
func (m MsgChangeAdmin) ValidateBasic() error {
...
_, err = sdk.AccAddressFromBech32(m.NewAdmin)
if err != nil {
return errorsmod.Wrapf(sdkerrors.ErrInvalidAddress, "Invalid addres
s (%s)", err)
}
...
}
```

`ValidateBasic()` is executed before the ante handler for standard transactions. As a result, any attempt to revoke admin rights by setting `NewAdmin = ""` fails during basic message validation, before the transaction can proceed further. Therefore, the documented “remove admin” flow cannot be executed through normal transactions.

This is a functional inconsistency rather than a direct security vulnerability. However, it has security-relevant implications: a token creator who intends to make a denom permanently immutable by removing its admin cannot do so through the standard transaction path.

Assets:

- x/tokenfactory [https://github.com/KiiChain/kiichain/tree/main/x/tokenfactory]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Severity: Low

Recommendations

Remediation: Update `MsgChangeAdmin.ValidateBasic()` so that `NewAdmin == ""` is treated as a valid input representing admin removal. Bech32 address validation should be applied only when a replacement admin is provided. This would align the implementation with the documented behavior of `ChangeAdmin`.

Resolution: The issue has been resolved in the referenced pull request.

Evidences

Test

Reproduce:

```
go test -v -tags=test -run "TestKeeperTestSuite/TestTF01_ChangeAdmi
nEmptyStringBlockedByFullPipeline" ./x/tokenfactory/keeper/... -cou
nt=1
```

```
package keeper_test

import (
"fmt"
"math/rand"
"time"

abci "github.com/cometbft/cometbft/abci/types"

"github.com/cosmos/cosmos-sdk/crypto/keys/secp256k1"
simtestutil "github.com/cosmos/cosmos-sdk/testutil/sims"
sdk "github.com/cosmos/cosmos-sdk/types"

"github.com/kiichain/kiichain/v7/x/tokenfactory/types"
)


func (suite *KeeperTestSuite) TestTF01_ChangeAdminEmptyStringBlocke
dByFullPipeline() {
suite.SetupTest()

app := suite.App
ctx := suite.Ctx
txConfig := app.GetTxConfig()
chainID := app.ChainID()

privKey := secp256k1.GenPrivKey()
senderAddr := sdk.AccAddress(privKey.PubKey().Address())

suite.FundAcc(senderAddr, sdk.NewCoins(
sdk.NewInt64Coin("stake", 1_000_000_000),
sdk.NewInt64Coin(types.DefaultParams().DenomCreationFee[0].Denom,
types.DefaultParams().DenomCreationFee[0].Amount.Int64()*10),
))

acc := app.AccountKeeper.NewAccountWithAddress(ctx, senderAddr)
app.AccountKeeper.SetAccount(ctx, acc)

res, err := suite.msgServer.CreateDenom(ctx, types.NewMsgCreateDeno
m(senderAddr.String(), "testdenom"))
suite.Require().NoError(err)
denom := res.GetNewTokenDenom()

accNum := acc.GetAccountNumber()
accSeq := acc.GetSequence()
nextHeight := app.LastBlockHeight() + 1

suite.Run("empty NewAdmin rejected by real tx pipeline", func() {
msg := types.NewMsgChangeAdmin(senderAddr.String(), denom, "")

signedTx, err := simtestutil.GenSignedMockTx(
```

```
rand.New(rand.NewSource(time.Now().UnixNano())),
txConfig,
[]sdk.Msg{msg},
sdk.NewCoins(sdk.NewInt64Coin("stake", 10000)),
simtestutil.DefaultGenTxGas,
chainID,
[]uint64{accNum},
[]uint64{accSeq},
privKey,
)
suite.Require().NoError(err)

txBytes, err := txConfig.TxEncoder()(signedTx)
suite.Require().NoError(err)

blockRes, err := app.FinalizeBlock(&abci.RequestFinalizeBlock{
Height: nextHeight,
Time: ctx.BlockTime().Add(5 * time.Second),
Txs: [][]byte{txBytes},
})
suite.Require().NoError(err)
suite.Require().Len(blockRes.TxResults, 1)

txResult := blockRes.TxResults[0]
suite.Require().NotEqual(uint32(0), txResult.Code,
"tx with empty NewAdmin should be rejected (non-zero code)")
suite.Require().Contains(txResult.Log, "Invalid address",
"rejection reason should mention invalid address")

fmt.Printf("TF-01 CONFIRMED via FinalizeBlock: tx rejected with cod
e=%d log=%q\n",
txResult.Code, txResult.Log)
})
}
```

Results:

```
=== RUN TestKeeperTestSuite
=== RUN TestKeeperTestSuite/TestTF01_ChangeAdminEmptyStringBlockedB
yFullPipeline
=== RUN TestKeeperTestSuite/TestTF01_ChangeAdminEmptyStringBlockedB
yFullPipeline/empty_NewAdmin_rejected_by_real_tx_pipeline
TF-01 CONFIRMED via FinalizeBlock: tx rejected with code=7 log="Inv
alid address (empty address string is not allowed): invalid address
"
--- PASS: TestKeeperTestSuite (0.09s)
--- PASS: TestKeeperTestSuite/TestTF01_ChangeAdminEmptyStringBlocke
dByFullPipeline (0.09s)
--- PASS: TestKeeperTestSuite/TestTF01_ChangeAdminEmptyStringBlocke
dByFullPipeline/empty_NewAdmin_rejected_by_real_tx_pipeline (0.00s)
PASS
ok github.com/kiichain/kiichain/v7/x/tokenfactory/keeper 0.147s
```

F-2026-15910 - CosmWasm Metadata Path Bypasses Capability Check - Low

Description:

Metadata updates are expected to be gated by `IsCapabilityEnabled` so they can be disabled when the corresponding capability is turned off.

However, the CosmWasm `PerformSetMetadata` implementation in *wasmbinding/tokenfactory/message_plugin.go* does not go through the tokenfactory message server. Instead, it calls the `bank.SetDenomMetaData` directly, which bypasses the capability check entirely.

This affects two execution paths:

- direct `SetMetadata` wasm messages
- `CreateDenom` wasm messages that include optional metadata

Because of this, capability checks are not applied uniformly between native and wasm flows. The current likelihood is low, but if the metadata support is disabled in a future upgrade, the restriction would still be ineffective for wasm-based calls.

Assets:

- Tokenfactory [https://github.com/KiiChain/kiichain/tree/main/wasmbinding/tokenfactory]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

Refactor `PerformSetMetadata` to delegate to `msgServer.SetDenomMetadata` instead of writing to the bank keeper directly. Centralizing metadata updates in the message server would ensure that capability checks, including `IsCapabilityEnabled`, are enforced consistently for both native and wasm-originated requests.

After this refactor, remove any validation and state-write logic in `PerformSetMetadata` that duplicates message-server behavior, such as admin

checks, metadata checks, and direct calls to the `bank.SetDenomMetaData`.

Introduce regression tests for both CosmWasm metadata paths to verify that capability gating behaves identically across native and wasm flows, including after the capability is disabled.

Resolution: The issue has been resolved in the referenced pull request.

F-2026-15503 - Vote Targets Are Not Set at Genesis - Info

Description:

The oracle uses a `VoteTarget` store to define which denoms validators are allowed to vote on. At genesis, this store is not initialized from `Params.Whitelist`. It only gets populated later when `ApplyWhitelist` runs in `EndBlocker`, and the first run happens at the end of the first vote period.

Because `VoteTarget` is empty during the first period (for a non-empty `Params.Whitelist`), `MsgAggregateExchangeRateVote` fails denom checks and returns `ErrUnknownDenom`. In simple terms, the oracle cannot accept votes for whitelisted denoms until after the first period-ending `EndBlocker` that runs `ApplyWhitelist`. If the whitelist is empty, there is nothing valid to vote on anyway, so this gap only matters when the whitelist has entries.

The oracle has a first-period liveness gap because no vote can pass denom validation before `VoteTarget` is initialized. Downstream modules that depend on fresh oracle prices may use stale values during that period. This is a correctness and initialization issue rather than a direct attacker-profit exploit. The failure is deterministic and happens on startup.

Assets:

- x/oracle [https://github.com/KiiChain/kiichain/tree/main/x/oracle]

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation:

Initialize the `VoteTarget` inside `InitGenesis` from `data.Params.Whitelist` by using the same behavior as `ApplyWhitelist,` so genesis and post-genesis behavior stay aligned, and valid votes are possible from block 0. For consistency, apply the same bank metadata registration rules at genesis for new whitelist denoms as `ApplyWhitelist` does after genesis, so denom handling does not diverge between the two paths.

Resolution:

The issue has been resolved in the referenced pull request.

F-2026-15675 - Release Proportion Exceeds 1 When Block Time Surpasses EndTime - Info

Description:

During reward calculation, the released portion is derived from the time elapsed since the last release relative to the total remaining distribution period:

```
// Calculate linear release proportion between 0 and 1
releaseProportion := math.LegacyNewDecFromBigInt(timeElapsed).Quo(m
ath.LegacyNewDecFromBigInt(totalDuration))
```

The inline comment states that `releaseProportion` should be between `0` and `1`. However, this assumption does not always hold. In particular, during the final reward distribution of a schedule, it is possible and expected that `timeElapsed > totalDuration`. In such cases, `releaseProportion` becomes greater than `1`.

At present, this does not appear to cause an incorrect result in `CalculateReward`, because the final reward amount is later capped by the remaining distributable balance:

```
amountToRelease = math.MinInt(amountToRelease, remaining.Amount)
```

Therefore, this is currently a correctness and code clarity issue rather than a security issue, but the misleading comment and unconstrained intermediate value may increase maintenance risk.

Assets:

- x/rewards [https://github.com/KiiChain/kiichain/tree/main/x/rewards]

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation:

Consider adding an early-return guard in `BeginBlocker` before calling `CalculateReward`, for example:

```
if !ctx.BlockTime().Before(schedule.EndTime) {
// release all remaining directly, then deactivate
}
```

This would make the terminal distribution case explicit and avoid relying on a `releaseProportion` value greater than `1` during the final release. The guard should also cover the case where `ctx.BlockTime()` is exactly equal to `schedule.EndTime`, not only when it is greater.

Resolution: The issue has been resolved in this pull request

F-2026-15685 - Nil Fee Tokens In Genesis Can Panic During Validation Or InitGenesis - Info

Description:

`GenesisState.Validate` iterates `gs.FeeTokens.Items` without checking whether `FeeTokens` is `nil`. In Go, accessing an `Items` on a `nil` `*FeeTokenMetadataCollection` panics. The same pattern exists in `InitGenesis` via `*gs.FeeTokens`. Genesis JSON that omits `fee_tokens` or sets it to `null` (depending on unmarshaling) can therefore crash the process during `ValidateGenesis` or `InitGenesis` instead of returning a clean validation error.

This creates an operational footgun for tooling and chain initialization. Validators or automation scripts importing such a genesis file may encounter an unexpected panic instead of a standard invalid-genesis error, which complicates automation, CI workflows, and recovery procedures. Although the default genesis uses a non-`nil` empty collection, making this unlikely in standard deployments, the issue may still affect custom genesis generators or partially constructed genesis files.

Assets:

- x/feeabstraction [https://github.com/KiiChain/kiichain/tree/main/x/feeabstraction]

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation:

1. In `(*GenesisState).Validate`, if `gs.FeeTokens == nil`, return `ErrInvalidFeeTokenMetadata` (or wrap `fmt.Errorf`) instructing callers to supply an empty collection.
2. In `InitGenesis`, if `gs.FeeTokens == nil`, treat as empty: `k.FeeTokens.Set(ctx, types.FeeTokenMetadataCollection{})` or allocate `&FeeTokenMetadataCollection{}` before dereferencing.
3. Add a unit test: `NewGenesisState(DefaultParams(), nil)` → `Validate()` returns error, no panic.

Resolution: The issue has been addressed in this pull request

F-2026-15959 - Missing Event Emission Across Core Modules - Info

Description:

All four modules in scope contain notable gaps in event emission, which makes certain state changes and on-chain actions harder to observe, monitor, and index.

The most visible gaps by module are as follows:

- `x/rewards` emits no events at all. As a result, `UpdateParams`, `FundPool`, and `ChangeSchedule` execute without event-level visibility, and reward distributions performed in `BeginBlocker` also remain invisible to event-based monitoring systems.
- `x/tokenfactory` does not emit events for governance-driven parameter updates in `UpdateParams`. Event coverage is also missing for the wasm-triggered `SetMetadata` operation in `wasmbinding/tokenfactory/message_plugin.go`.
- `x/feeabstraction` does not emit events for governance parameter updates or fee-token registry replacement. In addition, the module can disable itself silently by setting `params.Enabled = false` when the base token has no oracle price, and individual fee tokens can be auto-disabled when TWAP resolves to zero. In both cases, only logger output is produced, with no corresponding chain event.
- `x/oracle` does not emit events for governance parameter changes in `UpdateParams`.

This is not a direct security issue, but it reduces observability and weakens monitoring, alerting, indexing, and incident investigation. As a result, important state changes and module actions may be harder to detect or may go unnoticed entirely.

Assets:

- x/rewards [https://github.com/KiiChain/kiichain/tree/main/x/rewards]
- x/oracle [https://github.com/KiiChain/kiichain/tree/main/x/oracle]
- x/tokenfactory [https://github.com/KiiChain/kiichain/tree/main/x/tokenfactory]
- Tokenfactory [https://github.com/KiiChain/kiichain/tree/main/wasmbinding/tokenfactory]
- x/feeabstraction [https://github.com/KiiChain/kiichain/tree/main/x/feeabstraction]

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation: Re-evaluate the current event-emission approach across the affected modules and add events for material actions that are currently not observable through chain events. In particular, close the identified gaps around governance parameter updates, administrative operations, automated state transitions, and enforcement actions so that important module behavior is consistently visible to indexers, monitoring systems, and operators.

Resolution: This issue has been addressed in this pull request

Observation Details

.

Disclaimers

Hacken Disclaimer

The blockchain protocol given for audit has been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in the protocol source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other protocol statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of the blockchain protocol.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Blockchain protocols are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the protocol can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited blockchain protocol.

Appendix 1. Severity Definitions

Severity	Description
Critical	Vulnerabilities that can lead to a complete breakdown of the blockchain network's security, privacy, integrity, or availability fall under this category. They can disrupt the consensus mechanism, enabling a malicious entity to take control of the majority of nodes or facilitate 51% attacks. In addition, issues that could lead to widespread crashing of nodes, leading to a complete breakdown or significant halt of the network, are also considered critical along with issues that can lead to a massive theft of assets. Immediate attention and mitigation are required.
High	High severity vulnerabilities are those that do not immediately risk the complete security or integrity of the network but can cause substantial harm. These are issues that could cause the crashing of several nodes, leading to temporary disruption of the network, or could manipulate the consensus mechanism to a certain extent, but not enough to execute a 51% attack. Partial breaches of privacy, unauthorized but limited access to sensitive information, and affecting the reliable execution of smart contracts also fall under this category.
Medium	Medium severity vulnerabilities could negatively affect the blockchain protocol but are usually not capable of causing catastrophic damage. These could include vulnerabilities that allow minor breaches of user privacy, can slow down transaction processing, or can lead to relatively small financial losses. It may be possible to exploit these vulnerabilities under specific circumstances, or they may require a high level of access to exploit effectively.
Low	Low severity vulnerabilities are minor flaws in the blockchain protocol that might not have a direct impact on security but could cause minor inefficiencies in transaction processing or slight delays in block propagation. They might include vulnerabilities that allow attackers to cause nuisance-level disruptions or are only exploitable under extremely rare and specific conditions. These vulnerabilities should be corrected but do not represent an immediate threat to the system.

Appendix 2. Scope

The scope of the project includes the following components from the provided repository:

Scope Details	
Repository	https://github.com/KiiChain/kiichain/
Commit	cfe29972b490324c3a919bf44dd4212fc1564e1a

Appendix 3. Additional Valuables

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.